

Graylinx — On-Premise Deployment & Preparation Guide

This guide details the requirements, checklists, and step-by-step procedures to prepare and deploy the Graylinx AI Operations Intelligence Platform on-premise at a plant room or facility network.

1. Preparation Checklist

Before deploying the platform, verify that all hardware, software, network configurations, and database credentials on this checklist have been prepared.

Hardware Requirements

Depending on the deployment topology, you can run everything on a **single physical PC/Server box** (recommended for simplicity and lowest network latency) or separate the core app from the AI inference server.

Component	Minimum Requirement	Recommended Specification	Rationale
Graphics Card (GPU)	NVIDIA GPU with $\geq$ 12 GB VRAM (e.g., RTX 3060 12GB, RTX 4070 12GB)	NVIDIA GPU with $\geq$ 20 GB VRAM (e.g., RTX 4000 Ada, RTX 3090, RTX 4090)	Required to run qwen2.5:14b and RAG embedding models locally in VRAM. Less VRAM will cause models to spill to system RAM, causing extremely slow generation speeds.
Processor (CPU)	8-Core CPU (e.g., Intel Core i7, AMD Ryzen 7)	12 to 24-Core CPU (e.g., Intel Core Ultra 9 285K, Core i9, AMD Ryzen 9)	Runs concurrent API requests, Docker operations, and background anomaly scans.
System RAM	16 GB DDR4/DDR5 RAM	32 GB to 64 GB DDR5 RAM	Used by the host OS, dockerized databases (Postgres, Redis), API services, and active LLM layers.
Storage	512 GB SSD	1 TB NVMe SSD	Crucial for fast time-series database reads/writes and storing local LLM weight files.
Operating System	Windows 10/11 Pro or Ubuntu 20.04+	Ubuntu Server 22.04 / 24.04 LTS	Linux is the native environment for container operations in production.

Network Preparation

Graylinx is designed to run in a secure, air-gapped, or network-restricted environment. Configure the local network or VPN layer as follows:

[!IMPORTANT] The application uses **Tailscale** for secure overlay connections. Ensure the following network components are reachable:

- **MySQL Database Port (3307 or 3306):** The App Server must have network access to the plant's telemetry MySQL database.
- **Ollama API Port (11434):** The App Server must be able to send HTTP requests to the Ollama server.
- **Port Exposure (Client access):**
 - **:5173 or :80 / :443** (via Nginx reverse proxy) must be open to local plant operators/engineers so they can access the frontend web interface.

Database Preparation

Ensure you have the following database credentials and environments ready:

1. **Telemetry Source (MySQL):**
 - Read-only user credentials (for security, Graylinx never writes to the source telemetry database).
 - Database host IP and Port (configured in `.env` as `UNICHARM_DB_URL`).
2. **Application Database (PostgreSQL):**
 - Automatically run via the Docker Compose stack.
 - PostgreSQL must have the `pgvector` extension enabled (pre-configured in the `pgvector/pgvector:pg16` Docker image used in the stack).

2. Software Prerequisites

Install the following software packages on the target host machine(s):

- **Docker & Docker Compose:** To orchestrate all application services (Postgres, Redis, API, workers, and Nginx).
- **Tailscale:** To securely route traffic between the Graylinx server, the MySQL database, and the LLM engine.
- **Git:** To clone/pull updates of the repository.

3. Step-by-Step Deployment Flow

Once the checklist is ready, proceed with the deployment using these steps:

Step 1: Clone the Repository & Configure Environment

Clone the repository to the designated installation folder on the App Server:

```
git clone https://github.com/Kimosabey/thermynx.git
cd thermynx
```

```
cp .env.example .env
```

Edit the `.env` file and configure the target system parameters:

```
# Target Telemetry Database Connection (MySQL)
UNICHARM_DB_URL=mysql+aiomysql://ro_user:<password>@<db-host-ip>:3307/unicharm

# Local PostgreSQL Application Database
POSTGRES_URL=postgresql+asyncpg://thermynx:<prod-
password>@localhost:5432/thermynx_app
POSTGRES_PASSWORD=<prod-password>

# Local Redis Cache & Queue
REDIS_URL=redis://localhost:6379/0

# LLM Inference (Ollama Server)
OLLAMA_BASE_URL=http://<ollama-server-ip>:11434
OLLAMA_DEFAULT_MODEL=qwen2.5:14b

# CORS configuration (restrict to local origin)
CORS_ORIGINS=http://localhost:5173,http://<app-server-ip>:5173
```

Step 2: Initialize LLM Models (on the Ollama Server)

Access the Ollama Server (via terminal/SSH) and pull the required models:

```
# Pull default reasoning model (~9.0 GB)
ollama pull qwen2.5:14b

# Pull embedding model for vector storage/RAG (~274 MB)
ollama pull nomic-embed-text

# (Optional) Pull phi for fast anomaly narratives (~1.6 GB)
ollama pull phi:latest
```

Verify that the models are loaded:

```
ollama list
```

Step 3: Run the Docker Compose Stack

Start the base databases (Postgres with `pgvector` and Redis) on the App Server:

```
docker compose up -d
```

Verify that all services are running and healthy:

```
docker compose ps
```

Step 4: Setup Backend Dependencies & Run Migrations

Alembic will automatically handle Postgres schema updates. Setup your python virtual environment and apply migrations:

```
# Prepare virtual environment
python -m venv .venv
source .venv/bin/activate # On Windows: .venv\Scripts\activate

# Install dependencies
cd backend
pip install -r requirements.txt

# Run migrations to update Postgres schema
alembic upgrade head
```

Step 5: Ingest Initial Knowledge Base Documents (RAG)

Place any relevant equipment manuals, operational guidelines, or ASHRAE papers into the `docs/manuals/` directory, then run the bulk ingestion script:

```
python scripts/ingest_docs.py --dir ../docs/manuals
```

Step 6: Start Application Services

Start the backend API and the frontend client:

```
# In Terminal 1 (Backend API)
cd backend
uvicorn main:app --host 0.0.0.0 --port 8000

# In Terminal 2 (Frontend Client UI)
cd frontend
npm install
npm run dev -- --host
```

(Note: In production environments, Nginx is used to reverse-proxy port 80/443 directly to the frontend static build and proxy requests to `/api/*` to the FastAPI backend).

4. Verification and Health Checks

Verify that the system is fully operational by executing the following health endpoints:

1. General Liveness API

Run a curl command to check the API status:

```
curl http://localhost:8000/healthz
```

Expected output: `{"status": "ok"}`

2. Dependency Connectivity API

Check if the API server successfully connects to the telemetry database and the LLM engine:

```
curl http://localhost:8000/api/v1/health
```

Expected output:

```
{
  "status": "ok",
  "db": { "connected": true, "latency_ms": 15 },
  "ollama": { "connected": true, "default_model": "qwen2.5:14b", "latency_ms": 38 }
}
```

3. RAG Vector Database Ingestion Status

Verify that the uploaded manuals and guides are indexed in Postgres `pgvector`:

```
curl http://localhost:8000/api/v1/rag/status
```

Expected output: `{"ready": true, "total_chunks": 142, "sources": [...]}`

5. Future Expansion: Apache Kafka Considerations

If **Apache Kafka** is integrated in future phases (e.g., for high-throughput real-time telemetry ingestion, event queueing, or multi-agent coordination), the on-premise hardware resource footprint will increase.

Additional Hardware Requirements for Kafka:

- **System RAM:** Running a Zookeeper/KRaft controller and a Kafka Broker alongside local databases and Ollama will require dedicated heap memory.
 - *Minimum with Kafka:* **32 GB RAM**
 - *Recommended with Kafka:* **64 GB RAM** (to prevent OS swap latency).
- **Storage (SSD Mandatory):** Kafka logs messages sequentially to disk and relies heavily on high disk write I/O. **Mechanical HDDs are not supported.** Allocate an extra **100 GB to 200 GB of NVMe SSD storage** dedicated for Kafka data directory/log retention.
- **CPU Allocation:** A minimum of **12 logical cores** is recommended to handle messaging loops without interrupting background AI analyzer and anomaly detection tasks.